HS PF

Programmieren in Python
Datenaufbereitung für Machine Learning

Prof. Dr.-Ing. Stefan Kray

Daten für Machine Learning

- In der KI sind einige Datenformate üblich
- Eingangsdatensätze müssen häufig in diese Zielformate konvertiert werden

Klassen und Label

- Ein häufiges Anwendungsgebiet für Machine Learning ist das Klassifizieren von Daten
- z. B. sollen Hundebilder ausgewertet und der entsprechenden Hunderasse zugeordnet werden
- Man bezeichnet die verschiedenen "Kategorien", die auftreten können (hier die Hunderassen) als Klasse
- Label ist eine willkürliche Zuordnung einer Nummer zu jeder Klasse; repräsentiert die Klasse numerisch
- Beispiel:

Klasse	Label
Beagle	0
Golden Retriever	1
Collie	2
...	...

Stanford Dogs Dataset;
http://vision.stanford.edu/aditya86/ImageNetDogs/



Beagle



Golden Retriever

Features und Feature-Vektoren

- Messbare Eigenschaften eines Objekts werden „Feature“ genannt
- Beispiele Hunde:
  - Körpergröße
  - Fellfarbe
  - Größe der Ohren
  - Laufgeschwindigkeit
  - …
- Features werden in der Regel durch nummerische Werte ausgedrückt
- Die Sammlung aller Werte zu einem Klassifizierungs- oder Regressionsproblem wird „Feature-Vektor“ genannt
- **Ziel einer Klassifikation ist, für einen gegebenen Feature-Vektor das dazugehörige Label zu berechnen**

Feature-Vektoren (1)

- Beispiel:

Feature	Wert
Körpergröße	0,7
Fellfarbe	0,2
Ohrlänge	0,35
Laufge-schwindig-keit	0,8
…	…

Feature-Vektor

Feature-Vektor zu „Collie"

0,7
0,2
0,35
0,8
…



Label=2

ML-Algorithmus

Feature-Vektoren (2)

- Beispiel:

Feature	Wert
Körpergröße	0,7
Fellfarbe	0,2
Ohrlänge	0,35
Laufge-schwindig-keit	0,8
…	…

Feature-Vektor

Feature-Vektor zu „Collie"

0,7
0,2
0,35
0,8
…



Label=2

ML-Algorithmus

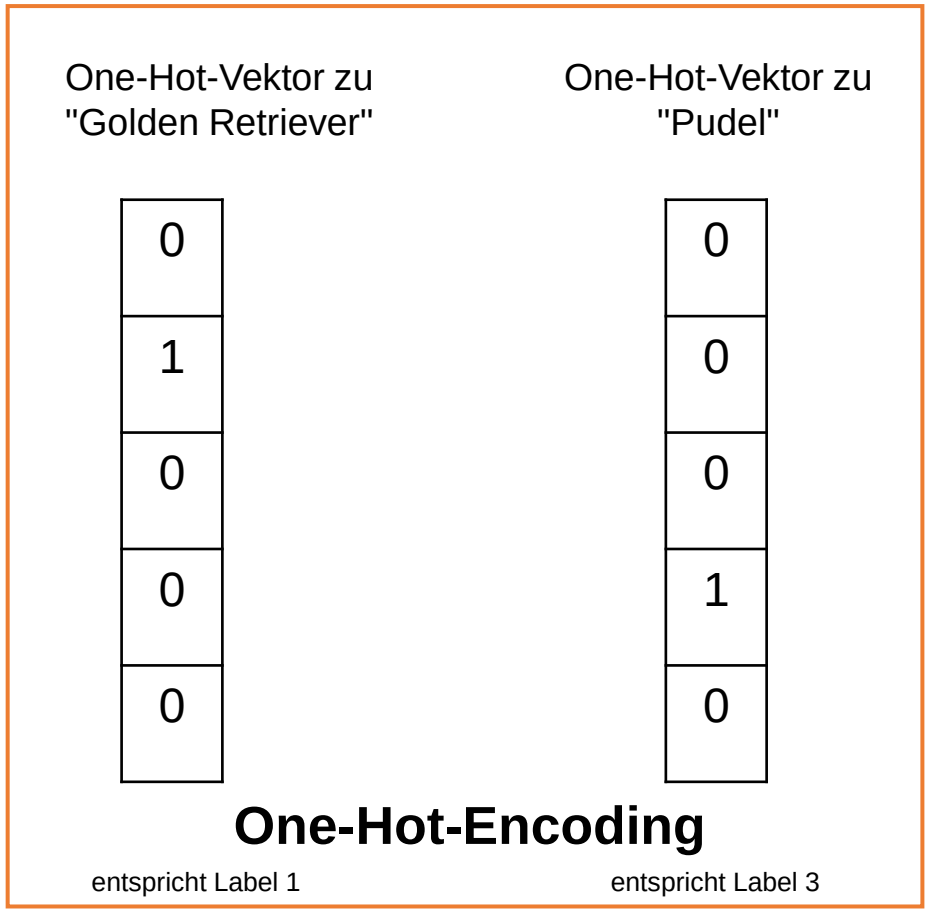
Problem: Wir wollen neben der Vorhersage auch die Wahrscheinlichkeit (Sicherheit) haben. → Eine Labelnummer als Ausgabe reicht nicht aus!

Lösung: One Hot Encoding

One-Hot-Encoding der Labels

- Die Zugehörigkeit zu Kategorien wird in der Regel „One-Hot“ kodiert
- Eine 1 im Vektor heißt zugehörig, eine 0 bedeutet nicht zugehörig
- Der Vektor ist so lang wie die Anzahl der zu unterscheidenden Klassen
- Beispiel:

Klasse	Label
Beagle	0
Golden Retriever	1
Collie	2
Pudel	3
Schäfer-hund	4



One-Hot-Encoding wird für das Training (Supervised Learning) benutzt!

Aufgabe onehot.py

- Öffnen Sie das Programm onehot.py
- Erzeugen Sie eine Liste von Klassen aus den Eingabedaten
- Erzeugen Sie für jedes Datum ein Label
- Erzeugen Sie für jedes Datum (Element der Liste data) einen One-Hot kodierten Vektor

- Hinweise:
  - Die Python-Funktion "set" erzeugt eine Menge aus einer Liste
  - Mit dem Befehl "list" machen Sie aus einem "set" wieder eine Liste
  - Eine Suche in einer Liste l lässt sich mit $l.index(\text{"}\ldots\text{"})$ durchführen

Interpretation One-Hot-Vektor zur Klassifikation (1)

- Annahme: ML-Algorithmus zur Klassifikation gibt One-Hot kodierte Vektoren aus
- Erwartung: Eine 1 an der Stelle der erkannte Klasse (z.B. Hunderasse)
- Was bedeutet folgender Vektor?

Klasse
Beagle
Golden Retriever
Collie
Pudel
Schäferhund

2
16
1
1
0

Klasse	Label
Beagle	0
Golden Retriever	1
Collie	2
Pudel	3
Schäfer-hund	4

Interpretation One-Hot-Vektor zur Klassifikation (2)

- Annahme: ML-Algorithmus zur Klassifikation gibt One-Hot kodierte Vektoren aus
- Erwartung: Eine 1 an der Stelle der erkannte Klasse (z.B. Hunderasse)
- Was bedeutet folgender Vektor

2
16
1
1
0

geteilt durch Summe aller Elemente:
$l = 2 + 16 + 1 + 1 = 20$



0.1
0.8
0.05
0.05
0

- → 80 % Wahrscheinlichkeit für Golden Retriever, 10 % Beagle, 5 % Collie, 5 % Pudel
- In der Praxis: Maximum des Vektors ist die gewählte Klasse

Beispiel: Prädiktion der Zugehörigkeit

- Beispiel:

Feature	Wert
Körpergröße	0,2
Fellfarbe	0,4
Ohrlänge	0,25
Laufge-schwindig-keit	0,6
…	…

Feature-Vektor

0,2
0,4
0,25
0,6
0



ML-Algorithmus

0,1
0,8
0,05
0,05
0

Klasse	Label
Beagle	0
Golden Retriever	1
Collie	2
Pudel	3
Schäfer-hund	4

Feature Vektor: Wertebereich

- Feature-Vektor
  - Vektor mit numerischen Werten, die eine Vielzahl von Parametern repräsentieren

0.2	Haarfarbe
2000	Einkommen
37	Alter
-3	Temperatur
23	Tag

Problem: Feature-Vektor Parameter

- Die Parameter eines Feature-Vektors können stark unterschiedliche Bereiche abbilden
- Gibt man diese Daten unverändert in einen Algorithmus, führen die starken Unterschiede zur (falschen) Betonung hoher Werte
- Eine Normierung der Parameterwerte ist notwendig, um eine gleichmäßige Gewichtung zu Gewährleisten
  - Idee: Mittelwert abziehen, durch Standardabweichung teilen
  - $x' \leftarrow \frac{x-\bar{x}}{\sigma}$
  - Rücktransformation: mit σ multiplizieren und $\bar{x}$ addieren.

Aufgabe normalize.py

- Laden Sie normalize.py
- Die Variable $data$ enthält Feature-Vektoren
- Normieren Sie die Feature-Vektoren $x_0 .. x_4$ entsprechend, d.h. berechnen Sie $data2$:
  - Berechnen Sie den Mittelwert Spaltenweise
  - Berechnen Sie die Standardabweichung Spaltenweise
  - Berechnen Sie $x_i' = \frac{x_i - \bar{x_i}}{\sigma_i}$

Sample	x0	x1	x2	x3	x4
0	6998	0,1361	0,3408	0,0000735	78596048
1	6580	0,4908	3,015	0,00004484	38462706
2	7563	0,9349	4,3465	0,00001003	6700340
3	8355	0,6529	2,1271	0,00002966	51430391
4	2393	0,4605	2,7561	0,00003395	27284192
5	9498	0,0244	2,7887	0,0000888	78543394
6	4030	0,6467	4,8231	0,00000403	19101443
7	5275	0,356	0,0705	0,00000899	96029352
8	8094	0,7979	3,9897	0,00006691	7307156
9	843	0,7892	0,9804	0,00005798	10179751
10	1221	0,9564	2,3944	0,00007815	14241835
11	5879	0,0329	2,0085	0,00009564	34243070
12	923	0,4159	1,7821	0,00002467	52404615
13	5882	0,0002	1,5362	0,00005066	18728752
14	1796	0,7247	2,319	0,00001332	96703562

Tabelle 4-4, S. 63

- Was gilt nun für den spaltenweisen Mittelwert und die spaltenweise Standardabweichung von $data2$?

Was passiert mit fehlenden Daten?

- Wenn nicht zu viele Daten fehlen, kann man Lücken durch den Mittelwert eines Features ersetzen

Sample	x0	x1	x2	x3	x4	
0	6998	0,1361	0,3408	0,0000735	78596048	
1	6580	0,4908	3,015	0,00004484	38462706	
2	7563	0,9349	4,3465	0,00001003	6700340	
3	8355	0,6529	2,1271	0,00002966	51430391	
4	2393	0,4605		0,00003395	27284192	
5		0,0244	2,7887	0,0000888	78543394	
6	4030	0,6467	4,8231	0,00000403	19101443	
7	5275	0,356	0,0705	0,00000899	96029352	
8	8094	0,7979	3,9897	0,00006691		
9	843	0,7892	0,9804	0,00005798	10179751	
10	1221	0,9564	2,3944	0,00007815	14241835	
11	5879	0,0329	2,0085	0,00009564	34243070	
12	923	0,4159	1,7821	0,00002467	52404615	
13	5882	0,0002	1,5362	0,00005066	18728752	
14	1796	0,7247	2,319	0,00001332	96703562	

Tabelle 4-4, S. 63

Was macht man mit fehlenden Daten?

- Wenn nicht zu viele Daten fehlen, kann man Lücken durch den Mittelwert eines Features ersetzen
- Dies beeinflusst eine nachfolgende Normierung nur in geringem Maße
- In dem Beispiel hat der neue Mittelwert ~ 5% Abweichung zu Originaldaten
- Je größer der Datensatz desto geringer der Effekt von einzelnen Lücken

Sample	x0	x1	x2	x3	x4
0	6998	0,1361	0,3408	0,0000735	78596048
1	6580	0,4908	3,015	0,00004484	38462706
2	7563	0,9349	4,3465	0,00001003	6700340
3	8355	0,6529	2,1271	0,00002966	51430391
4	2393	0,4605	2,3633	0,00003395	27284192
5	5074,62	0,0244	2,7887	0,0000888	78543394
6	4030	0,6467	4,8231	0,00000403	19101443
7	5275	0,356	0,0705	0,00000899	96029352
8	8094	0,7979	3,9897	0,00006691	43634373
9	843	0,7892	0,9804	0,00005798	10179751
10	1221	0,9564	2,3944	0,00007815	14241835
11	5879	0,0329	2,0085	0,00009564	34243070
12	923	0,4159	1,7821	0,00002467	52404615
13	5882	0,0002	1,5362	0,00005066	18728752
14	1796	0,7247	2,319	0,00001332	96703562

Tabelle 4-4, S. 63

Daten und überwachtes Lernen (1)

- Wir befassen uns zunächst mit überwachten Lernen (engl: supervised learning)
- D.h. dem ML-Algorithmus wird ein Eingabedatum gegeben, zugleich ist für das Datum ein "richtiges" Ergebnis bekannt
- Im Beispiel der Hunderassen: der Algorithmus erhält Bilder sowie die bekannte und korrekte Zuordnung zur Kategorie



Beagle

Golden Retriever





Training: Features + korrektes Label

Daten und überwachtes Lernen (2)

- Wir befassen uns zunächst mit überwachten Lernen (supervised learning)
- D.h. dem ML-Algorithmus wird ein Eingabedatum gegeben, zugleich ist für das Datum ein "richtiges" Ergebnis bekannt
- Im Beispiel der Hunderassen: der Algorithmus erhält Bilder sowie die bekannte und korrekte Zuordnung zur Kategorie



unbekannter Featurevektor







Golden Retriever

Vorhersage: Feature → Label

Daten und überwachtes Lernen (3)

- Wir befassen uns zunächst mit überwachten Lernen (supervised learning)
- D.h. dem ML-Algorithmus wird ein Eingabedatum gegeben, zugleich ist für das Datum ein "richtiges" Ergebnis bekannt
- Im Beispiel der Hunderassen: der Algorithmus erhält Bilder sowie die bekannte und korrekte Zuordnung zur Kategorie

- Wie viele Daten braucht man?
  - Antwort: so viele wie unter akzeptablen Aufwand möglich
  - Sollte in der Größenordnung einiger tausend Datensätze / Bilder liegen

Daten und überwachtes Lernen (4)

- Problem: wie bestimme ich, ob ML-Algorithmus "fertig" gelernt hat bzw. "besser" geworden ist?
- Idee: Validierung einschieben



Beagle

Golden Retriever

ML

Lernphase

Validierungsphase



bekannte Validierungsdaten; NICHT Teil des aktuellen Trainingszyklus!

Trefferquote: wie viele Validierungsdaten wurden richtig erkannt

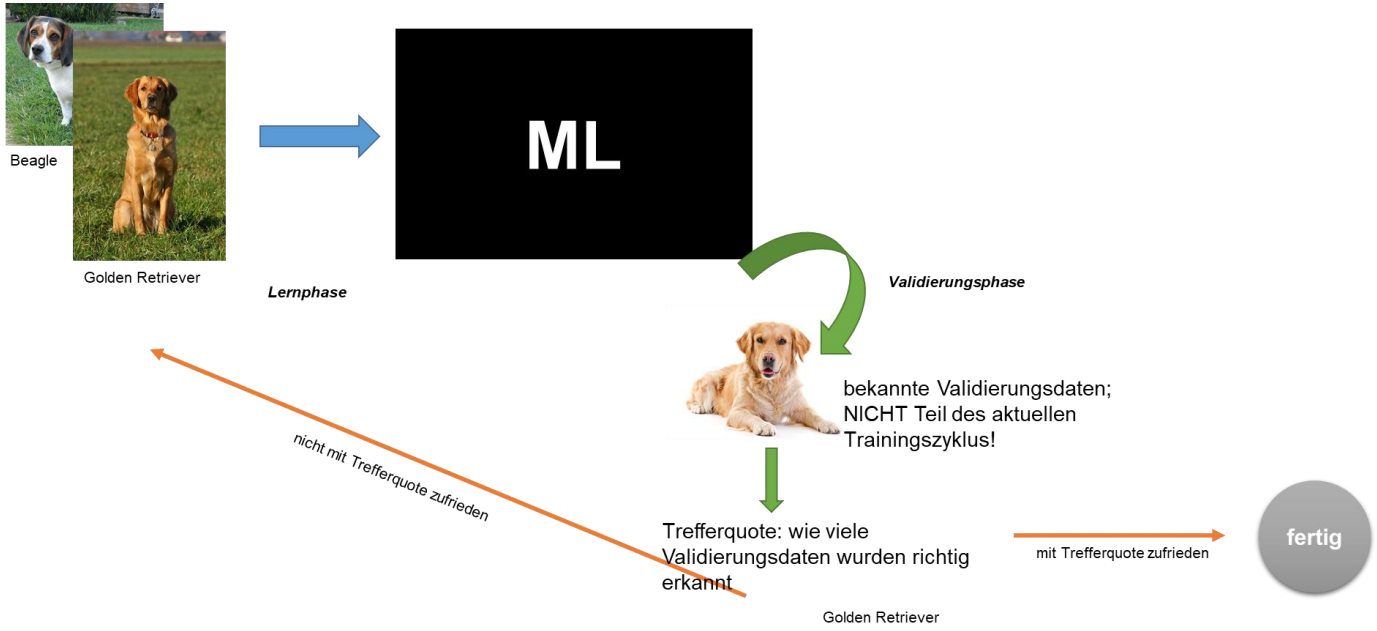
nicht mit Trefferquote zufrieden

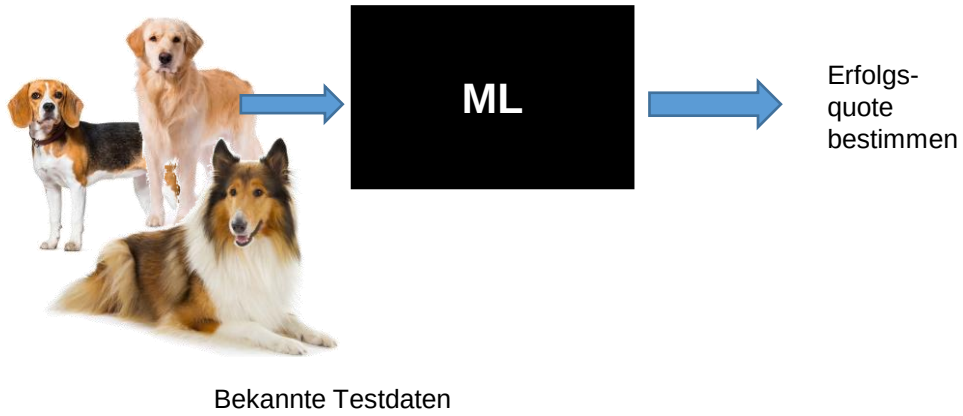
mit Trefferquote zufrieden

fertig

Daten und überwachtes Lernen (4)

- Problem: Validierungsdaten hängen über Rückkopplung im Algorithmus drin
- ML-Algorithmus könnte (langfristig) auf Validierungsdaten hintrainieren
- Lösung: einen dritten Datensatz zum Test zu nutzen, der weder Teil der Trainingsdaten, noch Teil der Validierungsdaten ist!
- Testdaten: Einmalig am Ende prüfen, ist nicht Teil einer "Feedback-Loop"





Aufteilung der Daten (1)

- Beim überwachten Lernen hat man gelabelte Daten
- Diese werden z.B. wie folgt aufgeteilt:
  - Train: 80 %; Validate: 10 %; Test: 10 %
  - üblich ist auch: Train: 90%; Validate: 5 %; Test: 5 %
  - oder auch: Train: 80 %; Test: 20 % (d. h. kein Validierungsschritt)



Quelle: https://medium.com/analytics-vidhya/a-simple-introduction-to-validating-and-testing-a-model-part-1-2a0765deb198

Aufgabe hunde.py

- Laden Sie hunde.py. Dort finden Sie 20 Hunde, die Sie in Trainingsdaten (80 %), Validierungsdaten (10%) und Testdaten (10 %) zufällig teilen sollen. Verwenden sie random.shuffle zum Mischen der Daten.

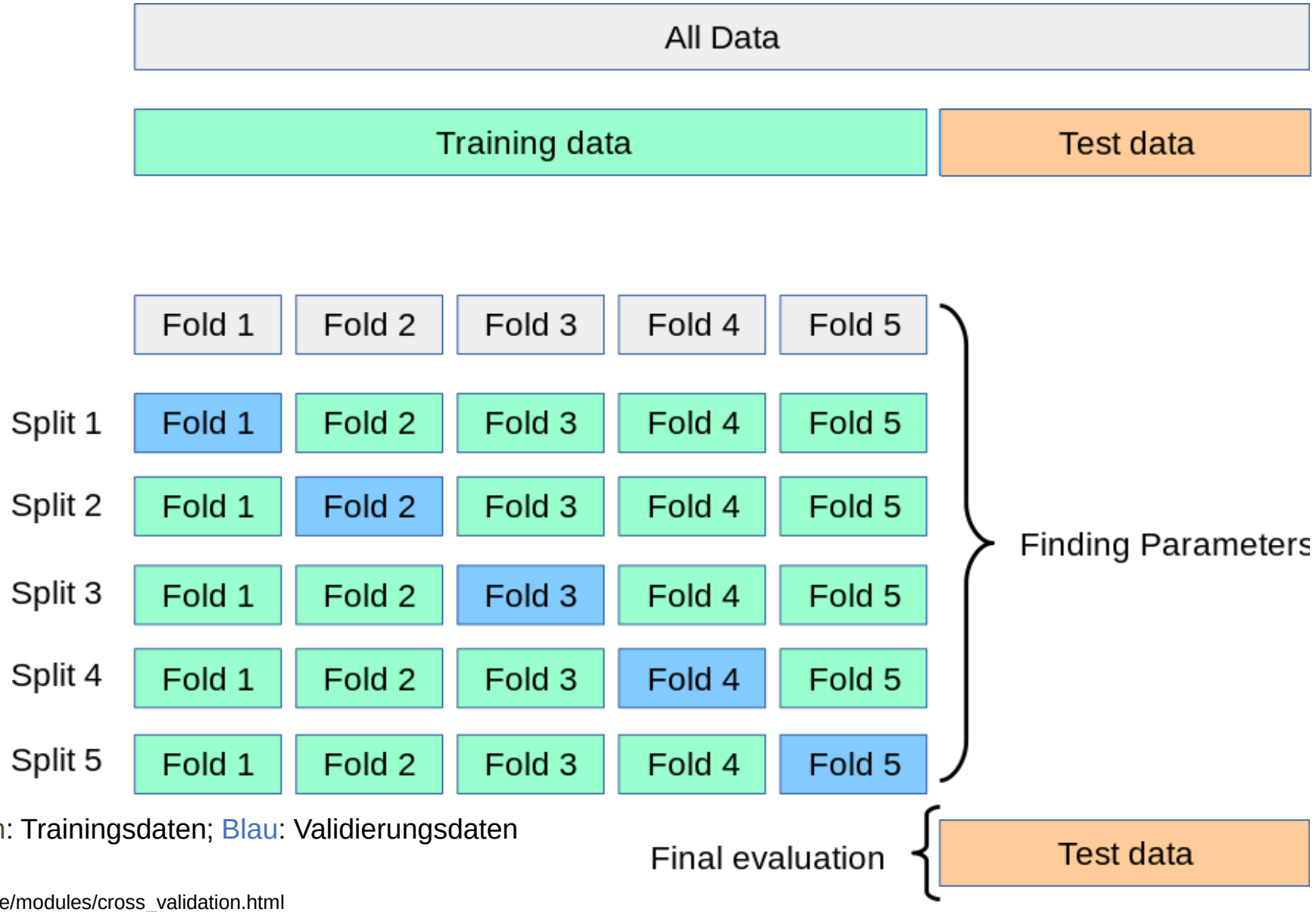
Aufteilung der Daten (2)

- Hold-out Validation: Daten zufällig aufteilen in *train* und *test.* Ersteren Datensatz weiter in *train* und *validation* aufspalten
  - Problem: Zufallsverteilung der Variablen in *train* und *test* kann Unterschiedlich sein. → Schlechter Lernerfolg möglich

- Stratified Hold-out Validation:
  - Jedes Teilset soll charakteristisch für den Datensatz sein. Die Daten werden so gesplittet, dass sie ähnliche Verteilungen (gemessen mit Mittelwert, Standardabweichung) haben
  - Problem: beide Varianten der Hold-out Validation reduzieren die Anzahl der Trainingsdaten

- k-Fold und stratified k-Fold Cross Validation
  - Datensatz in k-bins (Teil-Sets) teilen
  - 1 bin zufällig als Validierung-Datensatz wählen, die anderen als (k-1) als Trainingsdatensätze
  - das k Mal durchführen (und k Modelle anlernen); Ergebnisse mitteln
  - → alle Daten werden für das Training benutzt

k-Fold Cross validation



Grün: Trainingsdaten; Blau: Validierungsdaten

https://scikit-learn.org/stable/modules/cross_validation.html